



Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н. Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н. Э. Баумана)

ФАКУЛЬТЕТ «Информатика и системы управления (ИУ)»

КАФЕДРА «Информационные системы и телекоммуникации (ИУЗ)»

ОТЧЕТ

по лабораторной работе №3
по курсу «Разработка Программного обеспечения»
на тему: «Разработка frontend-приложения для REST API сервера
авторизации платежей транспортными картами»

Студент ИУЗ-61Б
(Группа)

(Подпись, дата)

Пялкин А. А.
(И. О. Фамилия)

Преподаватель

(Подпись, дата)

Сидякин И.М.
(И. О. Фамилия)

2026 г.

1 Теоретическая часть

1.1 Цель работы

Целью лабораторной работы является разработка frontend-приложения для REST API сервера авторизации платежей транспортными картами, реализованного в предыдущей лабораторной работе.

Frontend-приложение должно предоставлять графический интерфейс для авторизации пользователя, просмотра данных и выполнения CRUD-операций с основными сущностями системы.

1.2 Постановка задачи

В рамках лабораторной работы необходимо разработать web-интерфейс для backend-сервера авторизации платежей транспортными картами. Backend-сервер был реализован ранее и предоставляет REST API для работы с пользователями, терминалами, транспортными картами, ключами, транзакциями и терминальными операциями.

Frontend должен быть реализован с использованием React и TypeScript. Приложение должно содержать страницу входа, основную панель управления и страницы для работы с основными таблицами backend-сервера.

Для сущностей пользователей, терминалов, транспортных карт и ключей необходимо реализовать CRUD-интерфейс. Это означает, что пользователь должен иметь возможность просматривать список записей, создавать новые записи, редактировать существующие записи и удалять записи через соответствующие элементы интерфейса: формы, таблицы и кнопки действий.

Также необходимо реализовать интерфейс для терминальных функций backend-сервера: авторизации платежной транзакции и загрузки ключей. Frontend и backend должны запускаться из одного Docker-контейнера. В контейнере Nginx должен отдавать собранное React-приложение и проксировать REST API-запросы на Go backend.

1.3 Используемые технологии

Для реализации frontend-приложения используется библиотека React. Она позволяет строить пользовательский интерфейс из отдельных компонентов и обновлять страницу без полной перезагрузки.

TypeScript используется для статической типизации данных. Благодаря этому можно заранее описать структуры пользователей, терминалов, карт, ключей, транзакций и ответов backend-сервера.

Vite используется как инструмент сборки frontend-приложения. Он позволяет запускать проект в режиме разработки и собирать production-версию приложения в каталог **dist**.

Для маршрутизации внутри приложения используется библиотека **react-router-dom**. Она позволяет организовать отдельные страницы приложения: страницу входа, панель управления, страницы CRUD-операций и страницу терминальных инструментов.

Для взаимодействия frontend с backend используется REST API. Запросы выполняются к маршрутам с префиксом **/api/v1**. Для защищенных маршрутов frontend передает JWT-токен в заголовке **Authorization**.

Docker используется для сборки и запуска frontend и backend в одном контейнере. Nginx выполняет две функции: отдает статические файлы React-приложения и проксирует API-запросы на Go-сервер.

2 Практическая часть

2.1 Общая схема решения

В лабораторной работе была реализована клиентская часть системы авторизации платежей транспортными картами. Backend-сервер используется как готовый REST API, а frontend-приложение предоставляет пользователю графический интерфейс для работы с этим API.

Общая схема работы системы:

```
Browser
|
| HTTPS :8888
v
Nginx
|-- serves React frontend
|
| /api/v1 requests
v
Go backend server
|
v
SQLite database
```

Пользователь открывает приложение в браузере по адресу `https://localhost:8888`. Nginx отдает собранные статические файлы React-приложения. Если frontend выполняет запрос к `/api/v1`, Nginx проксирует его на Go backend, работающий внутри того же контейнера.

Таким образом frontend и backend доступны через один HTTPS-адрес, но выполняют разные функции: React отвечает за интерфейс, Go backend отвечает за бизнес-логику и работу с базой данных.

2.2 Структура проекта

Проект содержит backend-часть, frontend-часть, конфигурацию Nginx и файлы Docker-сборки. Backend расположен в каталогах `cmd`, `internal`, `migrations` и `docs`. Frontend расположен в каталоге `frontend`.

Основная структура проекта:

```
v7work/
|-- cmd/
```

```
|-- internal/
|-- migrations/
|-- docs/
|-- nginx/
|-- frontend/
|   |-- index.html
|   |-- package.json
|   |-- vite.config.ts
|   |-- src/
|       |-- api/
|       |-- components/
|       |-- contexts/
|       |-- layouts/
|       |-- pages/
|       |-- types/
|       |-- App.tsx
|       '-- main.tsx
|-- Dockerfile
|-- docker-compose.yml
|-- docker-entrypoint.sh
|-- go.mod
'-- go.sum
```

Каталог `frontend/src/pages` содержит страницы приложения. Каталог `frontend/src/components` содержит переиспользуемые компоненты интерфейса. Каталог `frontend/src/api` содержит код для отправки запросов к backend-серверу. Каталог `frontend/src/contexts` содержит контекст авторизации пользователя.

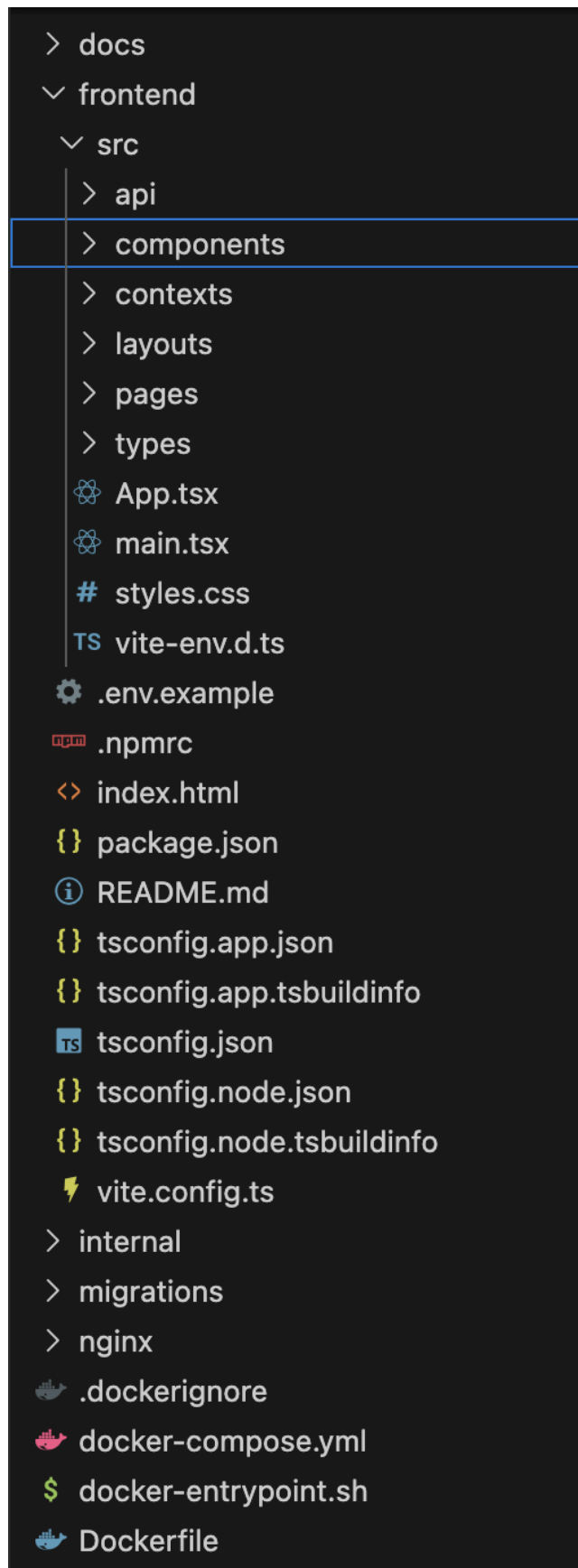


Рисунок 2.1 – Структура проекта с frontend-приложением

2.3 Backend

Backend-часть была реализована в предыдущей лабораторной работе и используется в данной работе как готовый REST API. Сервер предоставляет маршруты для авторизации пользователя, работы с пользователями, терминалами, транспортными картами, ключами, транзакциями и терминальными операциями.

В рамках лабораторной работы №3 backend не разрабатывался заново. Основная задача заключалась в разработке frontend-приложения, которое обращается к уже существующему API. Все защищенные запросы выполняются с передачей JWT-токена в заголовке Authorization.

Основные группы backend-маршрутов, используемые frontend-приложением:

```
POST /api/v1/auth/login

GET, POST, PUT, DELETE /api/v1/users
GET, POST, PUT, DELETE /api/v1/terminals
GET, POST, PUT, DELETE /api/v1/cards
GET, POST, PUT, DELETE /api/v1/keys

GET /api/v1/transactions
POST /api/v1/terminal/authorize
GET /api/v1/terminal/keys
```

2.4 Frontend

Frontend-часть реализована как одностраничное приложение на React и TypeScript. Для сборки проекта используется Vite.

В приложении реализованы следующие страницы:

```
/login
/
/users
/terminals
/cards
/keys
/transactions
/terminal-tools
```

Маршрут `/login` используется для страницы входа. Маршрут `/` открывает главную страницу приложения. Маршруты `/users`, `/terminals`, `/cards` и `/keys` используются для CRUD-операций. Маршрут `/transactions` используется для просмотра транзакций, а `/terminal-tools` — для терминальных инструментов.

Маршрутизация реализована с помощью `react-router-dom`. Страница входа доступна без авторизации, остальные страницы защищены компонентом `ProtectedRoute`. Если пользователь не авторизован, он перенаправляется на страницу входа.

Главный макет приложения реализован в компоненте `AppLayout`. Он содержит общую структуру интерфейса и навигацию между разделами приложения.

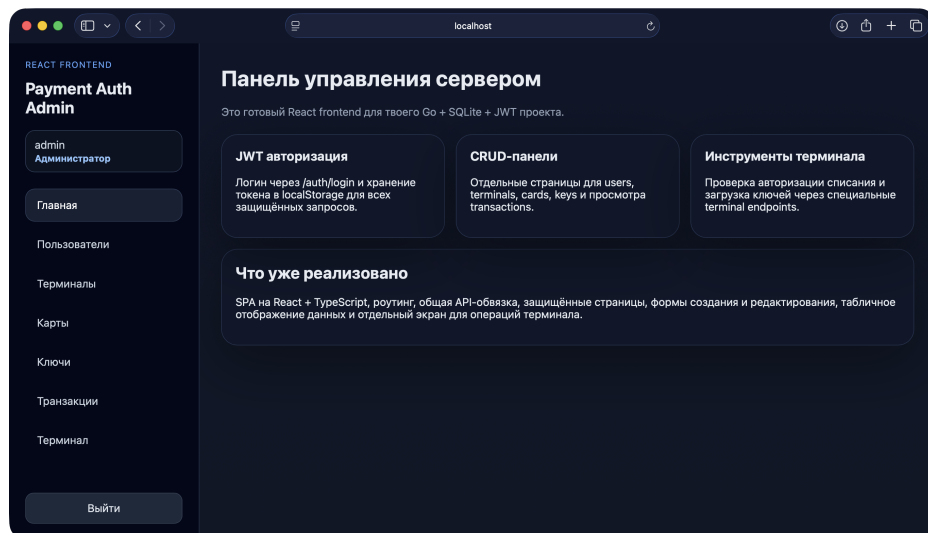


Рисунок 2.2 – Главная страница frontend-приложения

2.5 Авторизация пользователя

Авторизация пользователя выполняется на странице входа. Пользователь вводит логин и пароль, после чего frontend отправляет POST-запрос на маршрут backend-сервера:

```
POST /api/v1/auth/login
```

При успешной авторизации backend возвращает JWT-токен. Полученный токен сохраняется на стороне frontend и далее используется при обращении к защищенным маршрутам API.

Для хранения состояния авторизации используется `AuthContext`. Он

хранит информацию о текущем пользователе, токене и функциях входа и выхода из системы.

Пример заголовка авторизованного запроса:

```
Authorization: Bearer <JWT_TOKEN>
```

Если backend возвращает ошибку 401, frontend сообщает пользователю о необходимости повторного входа. Если backend возвращает ошибку 403, отображается сообщение о недостатке прав для выполнения операции.

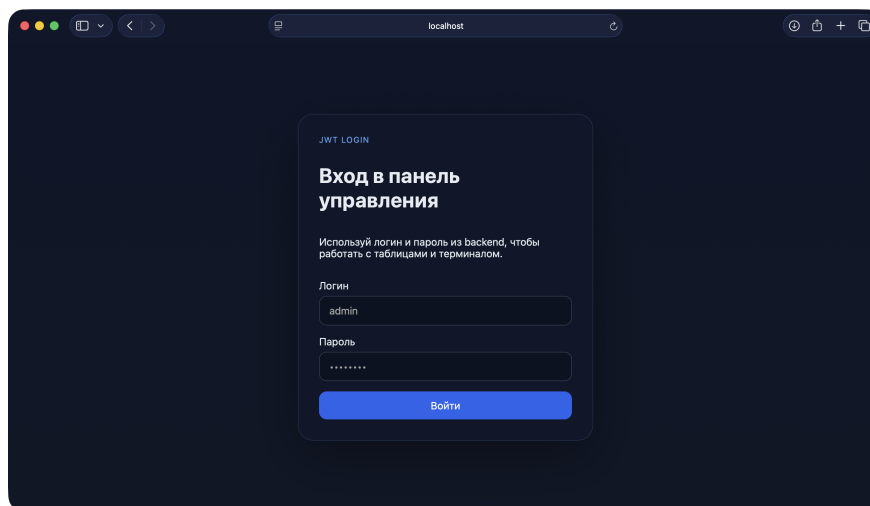


Рисунок 2.3 – Страница авторизации пользователя

2.6 API-клиент frontend-приложения

Для взаимодействия с backend был реализован общий API-клиент. Он находится в файле `frontend/src/api/client.ts`. API-клиент формирует HTTP-запросы, добавляет заголовки, сериализует тело запроса в JSON и обрабатывает ошибки ответа.

Базовый адрес API задается через переменную окружения `VITE_API_BASE_URL`. Если переменная не задана, используется значение по умолчанию:

```
/api/v1
```

API-клиент содержит методы для основных HTTP-операций:

```
apiClient.get()
apiClient.post()
apiClient.put()
apiClient.delete()
```

Поверх общего API-клиента реализованы сервисы для отдельных сущностей: пользователей, терминалов, карт, ключей, транзакций и терминальных операций. Такой подход позволяет не писать адреса API напрямую внутри компонентов страниц.

2.7 CRUD-интерфейс

Для пользователей, терминалов, транспортных карт и ключей был реализован CRUD-интерфейс. Он позволяет просматривать список записей, создавать новые записи, редактировать существующие записи и удалять записи.

Для реализации CRUD-страниц используется общий компонент `CrudSection`. Он получает описание колонок таблицы, описание полей формы и функции для работы с backend API. За счет этого один и тот же компонент можно использовать для разных сущностей.

Общий принцип работы CRUD-интерфейса следующий: сначала frontend загружает список записей через GET-запрос. Затем пользователь заполняет форму создания или редактирования. При сохранении frontend отправляет POST- или PUT-запрос, а при удалении — DELETE-запрос. После выполнения операции список записей обновляется повторным запросом к backend.

На каждой странице данные отображаются в таблице. Для администратора доступны кнопки создания, редактирования и удаления. Для обычного пользователя административные действия скрываются или блокируются, так как backend ограничивает права доступа.

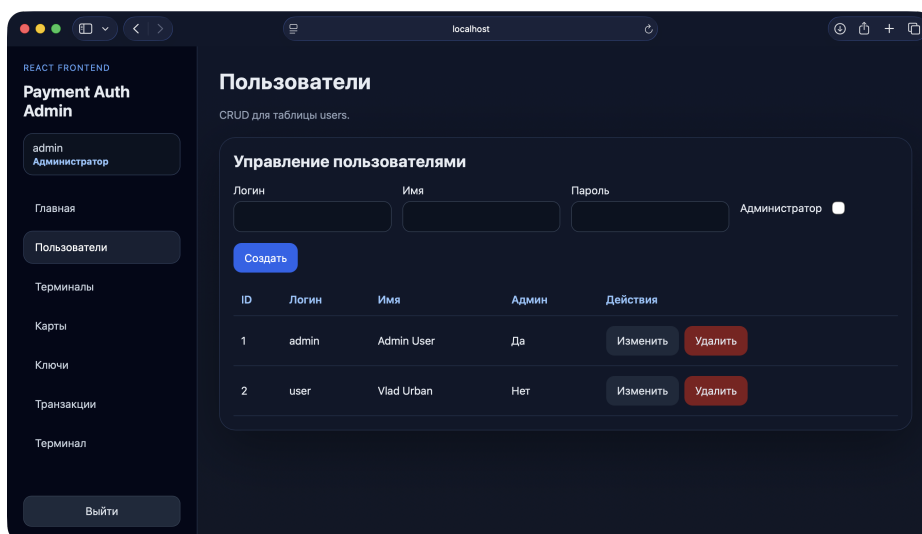


Рисунок 2.4 – CRUD-интерфейс для работы с пользователями

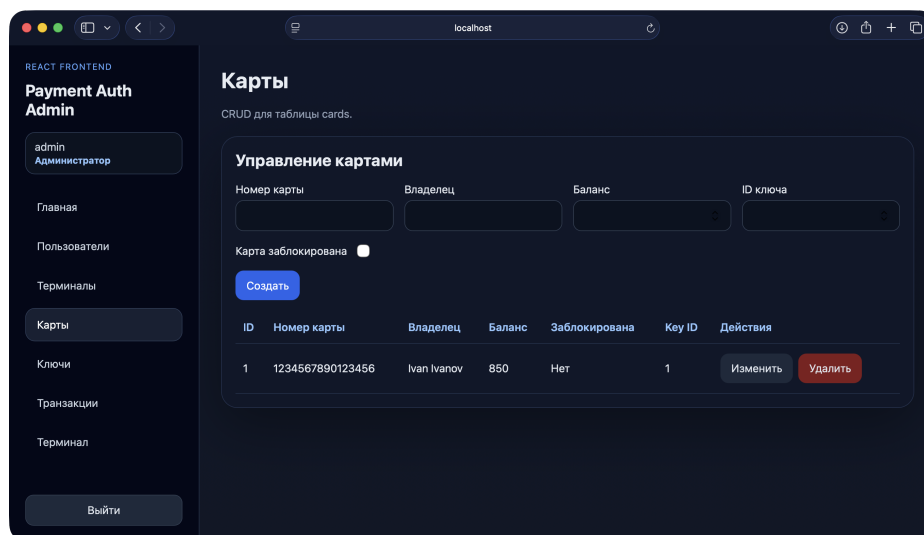


Рисунок 2.5 – CRUD-интерфейс для работы с транспортными картами

2.8 Интерфейс терминала

Для проверки специальных функций backend-сервера была реализована страница терминальных инструментов. Она доступна по маршруту:

```
/terminal-tools
```

На странице реализована форма авторизации платежной транзакции. Пользователь вводит номер карты, сумму операции и серийный номер терминала. После отправки формы frontend вызывает backend-маршрут:

```
POST /api/v1/terminal/authorize
```

Сервер проверяет карту, терминал, блокировку карты и достаточность средств. После проверки frontend отображает результат авторизации платежа.

Также на странице реализована загрузка ключей терминала. Для этого frontend отправляет GET-запрос:

```
GET /api/v1/terminal/keys
```

Полученные ключи отображаются в интерфейсе приложения.

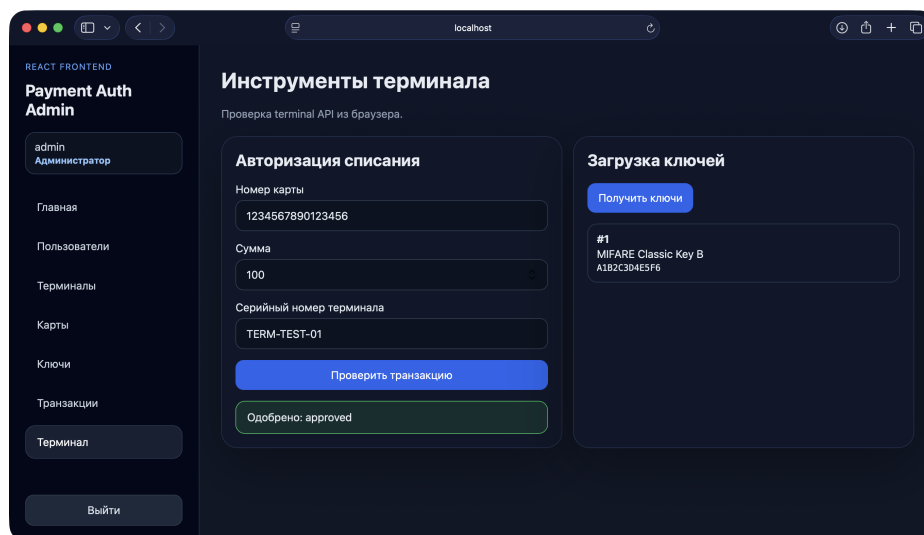


Рисунок 2.6 – Страница терминальных инструментов

2.9 Разграничение прав доступа

В системе используется ролевая модель доступа. Backend различает обычного пользователя и администратора. Frontend учитывает эту модель при отображении элементов интерфейса.

Администратор может выполнять операции создания, изменения и удаления данных. Обычный пользователь может просматривать данные, но не должен иметь доступа к административным операциям. Поэтому кнопки редактирования и удаления для обычного пользователя скрываются или не выполняют действие без прав.

Такое разграничение реализовано на двух уровнях. Во-первых, frontend скрывает недоступные действия в интерфейсе. Во-вторых, backend дополнительно проверяет права пользователя при выполнении защищенных запросов. Это необходимо, так как ограничения только на стороне frontend не являются достаточной защитой.

2.10 Клиент Flutter

Flutter-клиент в рамках лабораторной работы №3 не разрабатывался. В данной работе клиентская часть представлена web-приложением на React, которое взаимодействует с backend через REST API.

При этом backend сохраняет возможность подключения других клиентов, в том числе Flutter-приложения, так как обмен данными выполняется через HTTP-запросы в формате JSON.

2.11 Сборка frontend и backend в одном Docker-контейнере

Для запуска frontend и backend из одного Docker-контейнера был подготовлен многоэтапный Dockerfile.

На первом этапе используется образ Node.js. В нем устанавливаются зависимости frontend-приложения и выполняется сборка React-проекта:

```
npm install
npm run build
```

Результатом сборки является набор статических файлов в каталоге **dist**. На втором этапе используется образ Go. В нем загружаются зависимости backend-сервера и собирается исполняемый файл Go-приложения.

На финальном этапе создается итоговый образ, в который копируются backend-бинарник, миграции базы данных, собранный frontend и конфигурация Nginx. Nginx отдает React-приложение и проксирует API-запросы на Go backend.

Запуск контейнера выполняется командой:

```
docker compose up --build
```

После запуска frontend доступен по адресу:

```
https://localhost:8888/
```

A backend API доступен по адресу:

```
https://localhost:8888/api/v1
```

```
aleksandr@palkinMacBook-Pro-4 v7work % docker compose up --build
=> [internal] load metadata for docker.io/library/alpine:latest 0.0s
=> [internal] load metadata for docker.io/library/node:20-alpine 1.1s
=> [internal] load metadata for docker.io/library/golang:1.25-alpine 0.0s
=> [internal] load .dockerignore 0.0s
=> transferring context: 2kB 0.0s
=> [backend-builder 1/10] FROM docker.io/library/golang:1.25-alpine@sha256:8e02eb337d9e0ea459e041f1ee5eccc41cb61f1d83e7d883a3e2fb4862863fa 0.0s
=> resolve docker.io/library/golang:1.25-alpine@sha256:8e02eb337d9e0ea459e041f1ee5eccc41cb61f1d83e7d883a3e2fb4862863fa 0.0s
=> [internal] load build context 0.0s
=> transferring context: 3.00kB 0.0s
=> [stage-2 1/10] FROM docker.io/library/alpine:latest@sha256:25109184c71bdad752cd312a8c23239686a9b2071a8025f8aeb8f2198c3f659 0.0s
=> resolve docker.io/library/alpine:latest@sha256:25109184c71bdad752cd312a8c23239686a9b2071a8025f8aeb8f2198c3f659 0.0s
=> [frontend-builder 1/6] FROM docker.io/library/node:20-alpine@sha256:fb4cd12c85ee03686f6af5362a0b0d56d58c58a04632e6c0fb363f609372293 0.0s
=> resolve docker.io/library/node:20-alpine@sha256:fb4cd12c85ee03686f6af5362a0b0d56d58c58a04632e6c0fb363f609372293 0.0s
=> CACHED [stage-2 2/10] RUN apk add --no-cache add ca-certificates nginx openssl sqlite 0.0s
=> CACHED [stage-2 3/10] RUN mkdir -p /app/certs /app/data /usr/share/nginx/html /var/log/nginx /run/nginx 0.0s
=> CACHED [backend-builder 2/10] RUN apk add --no-cache gcc musl-dev 0.0s
=> CACHED [backend-builder 3/10] WORKDIR /app 0.0s
=> CACHED [backend-builder 4/10] COPY go.mod go.sum ./ 0.0s
=> CACHED [backend-builder 5/10] RUN go mod download 0.0s
=> CACHED [backend-builder 6/10] COPY cmd ./cmd 0.0s
=> CACHED [backend-builder 7/10] COPY internal ./internal 0.0s
=> CACHED [backend-builder 8/10] COPY docs ./docs 0.0s
=> CACHED [backend-builder 9/10] COPY migrations ./migrations 0.0s
=> CACHED [backend-builder 10/10] RUN CGO_ENABLED=1 GOOS=linux go build -a -installsuffix cgo -o server ./cmd/server 0.0s
=> CACHED [stage-2 4/10] COPY --from=backend-builder /app/server /app/server 0.0s
=> CACHED [stage-2 5/10] COPY --from=backend-builder /app/migrations /app/migrations 0.0s
=> CACHED [frontend-builder 2/6] WORKDIR /package 0.0s
=> CACHED [frontend-builder 3/6] COPY frontend/package.json ./ 0.0s
=> CACHED [frontend-builder 4/6] RUN npm config set registry https://registry.npmjs.org/ 0.0s
=> CACHED [frontend-builder 5/6] COPY frontend ./ 0.0s
=> CACHED [frontend-builder 6/6] RUN npm run build 0.0s
=> CACHED [stage-2 6/10] COPY --from=frontend-builder /frontend/dist /usr/share/nginx/html 0.0s
=> CACHED [stage-2 7/10] COPY nginx/nginx.conf /etc/nginx/nginx.conf 0.0s
=> CACHED [stage-2 8/10] COPY docker-entrypoint.sh /app/docker-entrypoint.sh 0.0s
=> CACHED [stage-2 9/10] RUN chmod +x /app/docker-entrypoint.sh 0.0s
=> CACHED [stage-2 10/10] RUN openssl req -x509 -nodes -days 365 -newkey rsa:2048 -keyout /app/certs/key.pem -out /app/certs/cert.pem -subj "/C=RU/ 0.0s
=> exporting to image 0.0s
=> exporting layers 0.0s
=> exporting manifest sha256:791845c9e9f9c7a7feca9f9186e9d5a106eb33c3dbef761c99b2a33d8744f 0.0s
=> exporting config sha256:1327a09f749f9367e8c14effdbbaf9626bda38e4c92db182d7dea3b41f 0.0s
=> exporting attestation manifest sha256:41a2001f9869b05fde6db19d337f7b00c040f9e5bbf1e98c5cc1054d85fa2e 0.0s
=> exporting manifest list sha256:357e56907a188428f6c0d0e0e10e049d5db300c5a2ed3186e5d0cc9293be 0.0s
=> naming to docker.io/library/v7work-payment-auth-server:latest 0.0s
=> unpacking to docker.io/library/v7work-payment-auth-server:latest 0.0s
=> resolving provenance for metadata file 0.0s
[+] up 3/0
✔ Image v7work-payment-auth-server Built 3.5s
✔ Network v7work_default Created 0.0s
✔ Container payment-auth-server Created 0.1s
```

Рисунок 2.7 – Запуск frontend и backend из одного Docker-контейнера

2.12 Вывод

В ходе выполнения лабораторной работы было разработано frontend-приложение для REST API сервера авторизации платежей транспортными картами. Были реализованы страница авторизации, защищенные маршруты, общий API-клиент, CRUD-интерфейс для основных сущностей, страница терминальных инструментов и учет прав пользователя в интерфейсе.

Также была реализована сборка frontend и backend в одном Docker-контейнере. Nginx используется для отдачи React-приложения и проксирования запросов к Go backend по единому HTTPS-адресу.

3 Список литературы

1. Документация React. Режим доступа: <https://react.dev/>
2. Документация TypeScript. Режим доступа: <https://www.typescriptlang.org/docs/>
3. Документация Vite. Режим доступа: <https://vite.dev/guide/>
4. Документация React Router. Режим доступа: <https://reactrouter.com/>
5. Документация Docker. Режим доступа: <https://docs.docker.com/>
6. Документация Nginx. Режим доступа: <https://nginx.org/en/docs/>
7. Документация JSON Web Token. Режим доступа: <https://jwt.io/introduction>